CHAPTER FIVE: PAGE LAYOUT: INTRODUCTION TO CSS (CASCADING STYLE SHEETS)



Learning Objectives:

By the end of this chapter the learner shall be able to;

i. Explain what is CSS ii. Create CSS document and format WebPages using CSS iii. Create CSS box model

5.1 Introduction to CSS

Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation semantics (the look and formatting) of a document written in a markup language. Its most common application is to style web pages written in HTML and XHTML, but the language can also be applied to any kind of XML document, including plain XML, SVG and XUL.

CSS is designed primarily to enable the separation of document content (written in HTML or a similar markup language) from document presentation, including elements such as the layout, colors, and fonts. This separation can improve content accessibility, provide more flexibility and control in the specification of presentation characteristics, enable multiple pages to share formatting, and reduce complexity and repetition in the structural content. CSS can also allow the same markup page to be presented in different styles for different rendering methods, such as on-screen, in print, by voice (when read out by a speech-based browser or screen reader) and on Braille-based, tactile devices. While the author of a document typically links that document to a CSS style sheet, readers can use a different style sheet, perhaps one on their own computer, to override the one the author has specified.

CSS specifies a priority scheme to determine which style rules apply if more than one rule matches against a particular element. In this so-called cascade, priorities or weights are calculated and assigned to rules, so that the results are predictable.

The CSS specifications are maintained by the World Wide Web Consortium (W3C). Internet media type (MIME type) text/css is registered for use with CSS by RFC 2318.

CSS Variations

CSS has various levels and profiles. Each level of CSS builds upon the last, typically adding new features and typically denoted as CSS 1, CSS 2, and CSS 3. Profiles are typically a subset of one or more levels of CSS built for a particular device or user interface. Currently there are profiles for mobile devices, printers, and television sets.

1. CSS 1

The first CSS specification to become an official W3C Recommendation is CSS level 1, published in December 1996. Among its capabilities are support for:

- Font properties such as typeface and emphasis
- Color of text, backgrounds, and other elements
- Text attributes such as spacing between words, letters, and lines of text
- Alignment of text, images, tables and other elements
- Margin, border, padding, and positioning for most elements
- Unique identification and generic classification of groups of attributes

2. CSS 2

CSS level 2 specification was developed by the W3C and published as a Recommendation in May 1998. A superset of CSS 1, CSS 2 includes a number of new capabilities like absolute, relative, and fixed positioning of elements and z-index, the concept of media types, support for aural style sheets and bidirectional text, and new font properties such as shadows.

CSS level 2 revision 1 or CSS 2.1 fixes errors in CSS 2, removes poorly-supported or not fully interoperable features and adds already-implemented browser extensions to the specification. In order to comply with the W3C Process for standardizing technical specifications, CSS 2.1 went back and forth between Working Draft status and Candidate Recommendation status for many years. CSS 2.1 first became a Candidate Recommendation on February 25, 2004, but it was reverted to a Working Draft on June 13, 2005 for further review. It was returned to Candidate Recommendation status on 19 July 2007 and was updated twice in 2009. However, since changes and clarifications were made to the prose it went back to Last Call Working Draft on 7 December 2010. Later it went into Proposed Recommendation on 12 April 2011. It was published as a Recommendation on 7 June 2011.

3. CSS 3

Instead of defining all features in a single, large specification like CSS 2, CSS 3 is divided into several separate documents called "modules". Each module adds new capability or extends features defined in CSS 2, over preserving backward compatibility. Work on CSS level 3 started around the time of publication of the original CSS 2 recommendation. The earliest CSS 3 drafts were published in June 1999.

Due to the modularization, different modules have different stability and are in different status. As of March 2011, there are over 40 CSS modules published from the CSS Working Group. Some modules such as Selectors, Namespaces, Color and Media Queries are considered stable and are either in Candidate Recommendation or Proposed Recommendation status. On 7 June 2011, the CSS 3 Color Module was published as a W3C Recommendation.

CSS Limitations

Some noted limitations of the current capabilities of CSS include:

- Poor controls for flexible layouts - While new additions to CSS 3 provide a stronger, more robust feature-set for layout, CSS is still at heart a styling language (for fonts, colours, borders and other decoration), not a layout language (for blocks with positions, sizes, margins, and so on). These limitations mean that creating fluid layouts generally requires hand-coding of CSS, and has held back the development of a standards-based WYSIWYG editor.
- Selectors are unable to ascend - CSS offers no way to select a parent or ancestor of an element that satisfies certain criteria. A more advanced selector scheme (such as XPath) would enable more sophisticated style sheets. However, the major reasons for the CSS Working Group rejecting proposals for parent selectors are related to browser performance and incremental rendering issues.
- Vertical control limitations - While horizontal placement of elements is generally easy to control, vertical placement is frequently unintuitive, convoluted, or impossible. Simple tasks, such as cantering an element vertically or getting a footer to be placed no higher than bottom of viewport, either require complicated and unintuitive style rules, or simple but widely unsupported rules.
- Absence of expressions - There is currently no ability to specify property values as simple expressions (such as margin-left: 10% – 3em + 4px;). This would be useful in a variety of cases, such as calculating the size of columns subject to a constraint on the sum of all columns. However, a working draft with a calc() value to address this limitation has been published by the CSS WG.

- Lack of column declaration - While possible in current CSS 3 (using the column-count module), layouts with multiple columns can be complex to implement in CSS 2.1. With CSS 2.1, the process is often done using floating elements, which are often rendered differently by different browsers, different computer screen shapes, and different screen ratios set on standard monitors.
- Cannot explicitly declare new scope independently of position - Scoping rules for properties such as z-index look for the closest parent element with a position:absolute or position:relative attribute. This odd coupling has undesired effects such as it is impossible to avoid declaring a new scope when one is forced to adjust an element's position, preventing one from using the desired scope of a parent element.
- Pseudo-class dynamic behavior not controllable - CSS implements pseudo-classes that allow a degree of user feedback by conditional application of alternate styles. One CSS pseudoclass, ":hover", is dynamic (equivalent of javascript "onmouseover") and has potential for abuse (e.g., implementing cursor-proximity popups), but CSS has no ability for a client to disable it (no "disable"-like property) or limit its effects (no "nochange"-like values for each property).

Advantages of CSS

- Flexibility - By combining CSS with the functionality of a Content Management System, a considerable amount of flexibility can be programmed into content submission forms. This allows a contributor, who may not be familiar or able to understand or edit CSS or HTML code to select the layout of an article or other page they are submitting on-the-fly, in the same form. When working with large-scale, complex sites, with many contributors such as news and informational sites, this advantage weighs heavily on the feasibility and maintenance of the project.
- Accessibility - Without CSS, web designers must typically lay out their pages with techniques that hinder accessibility for vision-impaired users, like HTML tables
- Separation of content from presentation - CSS facilitates publication of content in multiple presentation formats based on nominal parameters. Nominal parameters include explicit user preferences, different web browsers, the type of device being used to view the content (a desktop computer or mobile Internet device), the geographic location of the user and many other variables.
- Site-wide consistency - When CSS is used effectively, in terms of inheritance and "cascading," a global style sheet can be used to affect and style elements site-wide. If the situation arises that the styling of the elements should need to be changed or adjusted, these changes can be

made by editing rules in the global style sheet. Before CSS, this sort of maintenance was more difficult, expensive and time-consuming.

- Bandwidth - A stylesheet, whether internal to the source document or separate, will specify the style once for a range of HTML elements selected by class, type or relationship to others.

  This is much more efficient than repeating style information inline for each occurrence of the element. An external stylesheet is usually stored in the browser cache, and can therefore be used on multiple pages without being reloaded, further reducing data transfer over a network.
- Page reformatting - With a simple change of one line, a different style sheet can be used for the same page. This has advantages for accessibility, as well as providing the ability to tailor a page or site to different target devices. Furthermore, devices not able to understand the styling still display the content.

Styles Solved a Big Problem

HTML was never intended to contain tags for formatting a document. HTML was intended to define the content of a document, like: <h1>This is a heading</h1>

This is a paragraph.

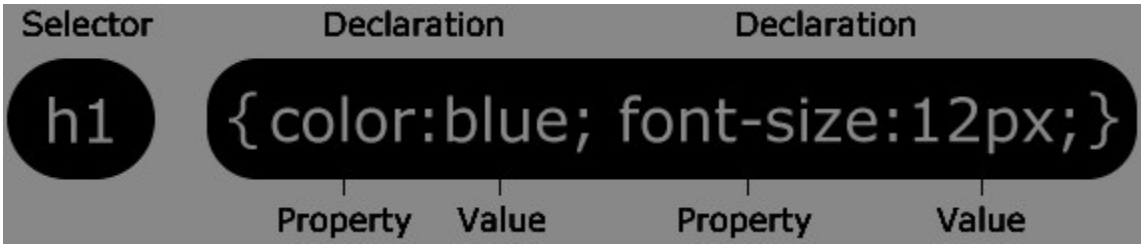
When tags like <font>, and color attributes were added to the HTML 3.2 specification, it started a nightmare for web developers. Development of large web sites, where fonts and color information were added to every single page, became a long and expensive process. To solve this problem, the World Wide Web Consortium (W3C) created CSS. In HTML 4.0, all formatting could be removed from the HTML document, and stored in a separate CSS file. All browsers support CSS today.

CSS Saves a Lot of Work

CSS defines HOW HTML elements are to be displayed. Styles are normally saved in external .css files. External style sheets enable you to change the appearance and layout of all the pages in a Web site, just by editing one single file!

CSS Syntax

A CSS rule has two main parts: a selector, and one or more declarations:



The *selector* is normally the HTML element you want to style.
Each *declaration* consists of a property and a value.

The *property* is the style attribute you want to change. Each property has a *value*.

CSS Example

A CSS declaration always ends with a semicolon, and declaration groups are surrounded by curly brackets:

p {color:red;text-align:center;}

To make the CSS more readable, you can put one declaration on each line, like this:

p { color:red;

text-

align:center;

}

CSS Comments

Comments are used to explain your code, and may help you when you edit the source code at a later date. Comments are ignored by browsers. A CSS comment begins with "/*", and ends with "*/", like this:

/*This is a

comment*/ p { text-

align:center;

/*This is another comment*/

color:black; font-

family:arial;

}

CSS Id and Class

In addition to setting a style for a HTML element, CSS allows you to specify your own selectors called "id" and "class".

The id Selector

The id selector is used to specify a style for a single, unique element. The id selector uses the id attribute of the HTML element, and is defined with a "#". The style rule below will be applied to the element with id="para1":

Example

#para1 { text-

align:center;

color:red; }

Note: Do NOT start an ID name with a number! It will not work in Mozilla/Firefox.

The class Selector

The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements. This allows you to set a particular style for many HTML elements with the same class. The class selector uses the HTML class attribute, and is defined with a "." In the example below, all HTML elements with class="center" will be center-aligned:

Example

.center {text-align:center;}

You can also specify that only specific HTML elements should be affected by a class. In the example below, all p elements with class="center" will be center-aligned:

Example

p.center {text-align:center;}

Note: Do NOT start a class name with a number! This is only supported in Internet Explorer.

Three Ways to Insert CSS

When a browser reads a style sheet, it will format the document according to it. There are three ways of inserting a style sheet:

1. External style sheet
2. Internal style sheet
3. Inline style

1. External Style Sheet

An external style sheet is ideal when the style is applied to many pages. With an external style sheet, you can change the look of an entire Web site by changing one file. Each page must link to the style sheet using the <link> tag. The <link> tag goes inside the head section:

<head>

<link rel="stylesheet" type="text/css" href="mystyle.css" />

</head>

An external style sheet can be written in any text editor. The file should not contain any html tags. Your style sheet should be saved with a .css extension. An example of a style sheet file is shown below:

hr {color:sienna;} p {margin-left:20px;} body

{background-image:url("images/back40.gif");}

2. Internal Style Sheet

An internal style sheet should be used when a single document has a unique style. You define internal styles in the head section of an HTML page, by using the <style> tag, like this:

<head>

<style type="text/css"> hr {color:sienna;} p

{margin-left:20px;} body {background-

image:url("images/back40.gif");} </style>

</head>

3. Inline Styles

An inline style loses many of the advantages of style sheets by mixing content with presentation. Use this method sparingly!

To use inline styles you use the style attribute in the relevant tag. The style attribute can contain any CSS property. The example shows how to change the color and the left margin of a paragraph:

This is a paragraph.

Multiple Style Sheets

If some properties have been set for the same selector in different style sheets, the values will be inherited from the more specific style sheet. For example, an external style sheet has these properties for the h3 selector:

h3 {

color:red;

text-

align:left;

font-size:8pt;

}

And an internal style sheet has these properties for the h3 selector:

h3 { text-

align:right;

font-size:20pt;

}

If the page with the internal style sheet also links to the external style sheet the properties for h3 will be:

color:red; text-

align:right; font-

size:20pt;

The color is inherited from the external style sheet and the text-alignment and the font-size is replaced by the internal style sheet.

Multiple Styles Will Cascade into One

Styles can be specified:

- inside an HTML element
- inside the head section of an HTML page
- in an external CSS file

Note: Even multiple external style sheets can be referenced inside a single HTML document.

Cascading order

What style will be used when there is more than one style specified for an HTML element? Generally speaking we can say that all the styles will "cascade" into a new "virtual" style sheet by the following rules, where number four has the highest priority:

1. Browser default
2. External style sheet
3. Internal style sheet (in the head section)
4. Inline style (inside an HTML element)

So, an inline style (inside an HTML element) has the highest priority, which means that it will override a style defined inside the <head> tag, or in an external style sheet, or in a browser (a default value).

Note: If the link to the external style sheet is placed after the internal style sheet in HTML <head>, the external style sheet will override the internal style sheet!

5.2 CSS Background

CSS background properties are used to define the background effects of an element. CSS properties used for background effects:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

Background Color

The background-color property specifies the background color of an element. The background color of a page is defined in the body selector: *Example* body {background-color:#b0c4de;}

With CSS, a color is most often specified by:

- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"
- a color name - like "red"

In the example below, the h1, p, and div elements have different background

colors: *Example* h1 {background-color:#6495ed;} p {background-color:#e0ffff;}

div {background-color:#b0c4de;}

Background Image

The background-image property specifies an image to use as the background of an element. By default, the image is repeated so it covers the entire element. The background image for a page can be set like this:

Example body {background-

image:url('paper.gif');}

Background Image - Repeat Horizontally or Vertically

By default, the background-image property repeats an image both horizontally and vertically. If the image is repeated only horizontally (repeat-x): *Example* body

{ background-

image:url('gradient2.png'); background-

repeat:repeat-x;

}

Background Image - Set position and no-repeat

Note: When using a background image, use an image that does not disturb the text. Showing the image only once is specified by the background-repeat property: *Example* body

{ background-

image:url('img_tree.png'); background-

repeat:no-repeat;

}

The position of the image is specified by the background-position property:

Example
body

{ background-

image:url('img_tree.png'); background-

repeat:no-repeat; background-

position:right top;

}

Background - Shorthand property

As you can see from the examples above, there are many properties to consider when dealing with backgrounds. To shorten the code, it is also possible to specify all the properties in one single property. This is called a shorthand property. The shorthand property for background is simply "background":

Example body {background:#ffffff url('img_tree.png') no-repeat right top;}

When using the shorthand property the order of the property values are:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

It does not matter if one of the property values is missing, as long as the ones that are present are in this order.

5.3 CSS Text

Text Color

The color property is used to set the color of the text. The default color for a page is defined in the body selector.

Example body

{color:blue;} h1

{color:#00ff00;} h2

{color:rgb(255,0,0);}

Note: For W3C compliant CSS: If you define the color property, you must also define the background-color property.

Text Alignment

The text-align property is used to set the horizontal alignment of a text. Text can be centered, or aligned to the left or right, or justified. When text-align is set to "justify", each line is stretched so that every line has equal width, and the left and right margins are straight (like in magazines and newspapers).
Example

h1 {text-align:center;}

p.date {text-align:right;}

p.main {text-align:justify;}

Text Decoration

The text-decoration property is used to set or remove decorations from text. The text-decoration property is mostly used to remove underlines from links for design purposes: *Example* a {text-decoration:none;}

It can also be used to decorate

text: *Example* h1 {text-

decoration:overline;} h2 {text-

decoration:line-through;} h3

{text-decoration:underline;}

h4 {text-decoration:blink;}

Note: It is not recommended to underline text that is not a link, as this often confuses users.

Text Transformation

The text-transform property is used to specify uppercase and lowercase letters in a text. It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word.

Example

p.uppercase {text-transform:uppercase;}

p.lowercase {text-transform:lowercase;}

p.capitalize {text-transform:capitalize;}

Text Indentation

The text-indentation property is used to specify the indentation of the first line of a

text. Example p {text-indent:50px;}

5.4 CSS Font

CSS font properties define the font family, boldness, size, and the style of a text.



On computer screens, sans-serif fonts are considered easier to read than serif fonts.

CSS Font Families

In CSS, there are two types of font family names:

- generic family - a group of font families with a similar look (like "Serif" or "Monospace")
- font family - a specific font family (like "Times New Roman" or "Arial")

Generic family	**Font family**	**Description**
Serif	Times New Roman Georgia	Serif fonts have small lines at the ends on some characters
Sans-serif	Arial Verdana	"Sans" means without - these fonts do not have the lines at the ends of characters

Monospace	Courier New Lucida Console	All monospace characters have the same width

Font Family

The font family of a text is set with the font-family property. The font-family property should hold several font names as a "fallback" system. If the browser does not support the first font, it tries the next font. Start with the font you want, and end with a generic family, to let the browser pick a similar font in the generic family, if no other fonts are available.

Note: If the name of a font family is more than one word, it must be in quotation marks, like fontfamily: "Times New Roman".

More than one font family is specified in a comma-separated

list: *Example* p{font-family:"Times New Roman", Times,

serif;}

For more commonly used font combinations, look at our Web Safe Font Combinations.

Font Style

The font-style property is mostly used to specify italic text. This property has three values:

- normal - The text is shown normally
- italic - The text is shown in italics
- oblique - The text is "leaning" (oblique is very similar to italic, but less supported)

Example

p.normal {font-style:normal;}

p.italic {font-style:italic;}
p.oblique {font-style:oblique;}

Font Size

The font-size property sets the size of the text. Being able to manage the text size is important in web design. However, you should not use font size adjustments to make paragraphs look like headings, or headings look like paragraphs. Always use the proper HTML tags, like <h1> - <h6> for headings and <p> for paragraphs. The font-size value can be an absolute, or relative size.

Absolute size:

- Sets the text to a specified size
- Does not allow a user to change the text size in all browsers (bad for accessibility reasons)

Absolute size is useful when the physical size of the output is known

Relative size:

- Sets the size relative to surrounding elements
- Allows a user to change the text size in browsers

Note: If you do not specify a font size, the default size for normal text, like paragraphs, is 16px (16px=1em).

Set Font Size With Pixels

Setting the text size with pixels, gives you full control over the text

size: Example h1 {font-size:40px;} h2 {font-size:30px;} p {font-

size:14px;}

The example above allows Firefox, Chrome, and Safari to resize the text, but not Internet Explorer.

The text can be resized in all browsers using the zoom tool (however, this resizes the entire page, not just the text).

Set Font Size With Em

To avoid the resizing problem with Internet Explorer, many developers use em instead of pixels. The em size unit is recommended by the W3C. 1em is equal to the current font size. The default text size in browsers is 16px. So, the default size of 1em is 16px. The size can be calculated from pixels to em using this formula: pixels/16=em *Example* h1 {font-size:2.5em;} /* 40px/16=2.5em */ h2 {font-size:1.875em;} /* 30px/16=1.875em */ p {font-size:0.875em;} /* 14px/16=0.875em */

In the example above, the text size in em is the same as the previous example in pixels. However, with the em size, it is possible to adjust the text size in all browsers.

5.5 CSS Links

Links can be styled in different ways. Links can be styled with any CSS property (e.g. color, fontfamily, background, etc.). Special for links are that they can be styled differently depending on what state they are in. The four links states are:

- a:link - a normal, unvisited link
- a:visited - a link the user has visited
- a:hover - a link when the user mouses over it
- a:active - a link the moment it is clicked *Example*

```
a:link {color:#FF0000;}      /* unvisited link */
a:visited {color:#00FF00;}  /* visited link */
a:hover {color:#FF00FF;}  /* mouse over link */
a:active {color:#0000FF;}  /* selected link */
```

When setting the style for several link states, there are some order rules:

- a:hover MUST come after a:link and a:visited
- a:active MUST come after a:hover

Common Link Styles

In the example above the link changes color depending on what state it is in.

Text Decoration

The text-decoration property is mostly used to remove underlines from links:

Example

```
a:link {text-
decoration:none;} a:visited {text-
decoration:none;} a:hover {text-
decoration:underline;} a:active
{text-decoration:underline;}
```

Background Color

The background-color property specifies the background color for links: Example

```
a:link {background-color:#B2FF99;}
a:visited {background-color:#FFFF85;}
```

a:hover {background-color:#FF704D;}

a:active {background-color:#FF704D;}

5.6 CSS Lists

The CSS list properties allow you to:

- Set different list item markers for ordered lists
- Set different list item markers for unordered lists
- Set an image as the list item marker

List

In HTML, there are two types of lists:

- unordered lists - the list items are marked with bullets
- ordered lists - the list items are marked with numbers or letters

With CSS, lists can be styled further, and images can be used as the list item marker.

Different List Item Markers

The type of list item marker is specified with the list-style-type property: Example

ul.a {list-style-type: circle;} ul.b

{list-style-type: square;} ol.c

{list-style-type: upper-roman;}

ol.d {list-style-type: lower-alpha;}

An Image as The List Item Marker

To specify an image as the list item marker, use the list-style-image property:

Example

ul

{

list-style-image: url('sqpurple.gif');

}

The example above does not display equally in all browsers. IE and Opera will display the imagemarker a little bit higher than Firefox, Chrome, and Safari.

If you want the image-marker to be placed equally in all browsers, a crossbrowser solution is explained below. *Example*

```
ul { list-style-type:
none; padding:
0px; margin: 0px;
}
li
{ background-image:
url(sqpurple.gif); background-
repeat: no-repeat; background-
position: 0px 5px; padding-left:
14px;
}
```

Example explained:

- For ul:
  - Set the list-style-type to none to remove the list item marker
  - Set both padding and margin to 0px (for cross-browser compatibility)
- For li:
  - Set the URL of the image, and show it only once (no-repeat)
  - Position the image where you want it (left 0px and down 5px)
  - Position the text in the list with padding-left

List - Shorthand property

It is also possible to specify all the list properties in one, single property. This is called a shorthand property. The shorthand property used for lists, is the list-style property: Example ul {

```
list-style: square url("sqpurple.gif");
}
```

When using the shorthand property, the order of the values are:

- list-style-type
- list-style-position
- list-style-image

It does not matter if one of the values above are missing, as long as the rest are in the specified

5.7 CSS Tables

The look of an HTML table can be greatly improved with CSS.

Table Borders

To specify table borders in CSS, use the border property. The example below specifies a black border for table, th, and td elements: Example

```
table, th, td
{ border: 1px solid
black;
}
```

Notice that the table in the example above has double borders. This is because both the table, th, and td elements have separate borders. To display a single border for the table, use the border-collapse property.

Collapse Borders

The border-collapse property sets whether the table borders are collapsed into a single border or separated: *Example*

```
table { border-
collapse:collapse;
}
table,th, td
{
border: 1px solid black;
}
```

Table Width and Height

Width and height of a table is defined by the width and height properties. The example below sets the width of the table to 100%, and the height of the th elements to 50px: *Example*

```
table {
width:100
%;
} th {
height:50p
x;
}
```

Table Text Alignment

The text in a table is aligned with the text-align and vertical-align properties. The text-align property sets the horizontal alignment, like left, right, or center: *Example*

```
td { text-
align:right;
}
```

The vertical-align property sets the vertical alignment, like top, bottom, or middle: *Example*

```
td { height:50px;
vertical-
align:bottom;
}
```

Table Padding

To control the space between the border and content in a table, use the padding property on td and th elements:

Example

```
td {
padding:15p
x;
}
```

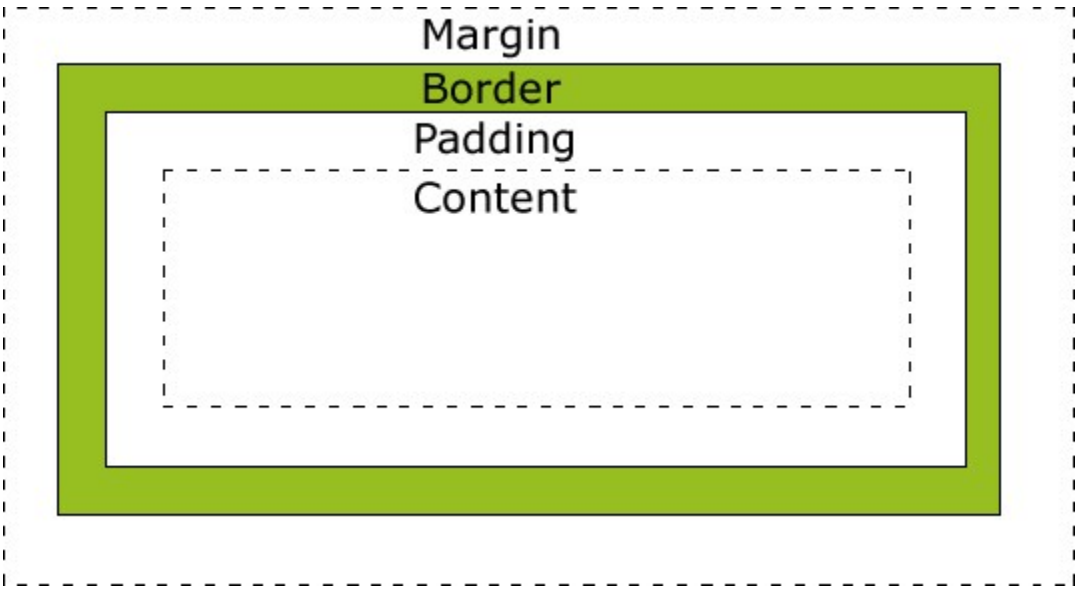
Table Color

The example below specifies the color of the borders, and the text and background color of th *elements*: Example

```
table, td, th
{
border:1px solid green;
} th { background-
color:green;
color:white;
}
```

5.8 The CSS Box Model

All HTML elements can be considered as boxes. In CSS, the term "box model" is used when talking about design and layout. The CSS box model is essentially a box that wraps around HTML elements, and it consists of: margins, borders, padding, and the actual content. The box model allows us to place a border around elements and space elements in relation to other elements. The image below illustrates the box model:



Explanation of the different parts:

- Margin - Clears an area around the border. The margin does not have a background color, it is completely transparent
- Border - A border that goes around the padding and content. The border is affected by the background color of the box
- Padding - Clears an area around the content. The padding is affected by the background color of the box
- Content - The content of the box, where text and images appear

In order to set the width and height of an element correctly in all browsers, you need to know how the box model works.

Width and Height of an Element

When you specify the width and height properties of an element with CSS, you are just setting the width and height of the content area. To know the full size of the element, you must also add the padding, border and margin. The total width of the element in the example below is 300px:

width:250px;

padding:10px;

border:5px solid gray;

margin:10px;

Let's do the math:

250px (width)

+ 20px (left and right padding)

+ 10px (left and right border)

+ 20px (left and right margin)

= 300px

Imagine that you only had 250px of space. Let's make an element with a total width of 250px: *Example* width:220px; padding:10px; border:5px solid gray; margin:0px;

The total width of an element should always be calculated like this:

Total element width = width + left padding + right padding + left border + right border + left margin + right margin

The total height of an element should always be calculated like this:

Total element height = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin ***Browsers Compatibility Issue***

If you tested the previous example in Internet Explorer, you saw that the total width was not exactly 250px. IE8 and earlier versions includes padding and border in the width, when the width property is set, unless a DOCTYPE is declared. To fix this problem, just add a DOCTYPE to the code:

Example

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html>
<head>
<style
type="text/css">
```

div.ex { width:220px;

padding:10px;

border:5px solid gray;

margin:0px;

}

</style>

</head>

CSS Border Properties

The CSS border properties allow you to specify the style and color of an element's border.

Border Style

The border-style property specifies what kind of border to display. None of the border properties will have ANY effect unless the border-style property is set. border-style values:

none: Defines no border

dotted: Defines a dotted border

dashed: Defines a dashed border

solid: Defines a solid border

double: Defines two borders. The width of the two borders are the same as the border-width value

groove: Defines a 3D grooved border. The effect depends on the border-color value

ridge: Defines a 3D ridged border. The effect depends on the border-color value

inset: Defines a 3D inset border. The effect depends on the border-color value

outset: Defines a 3D outset border. The effect depends on the border-color value

Border Width

The border-width property is used to set the width of the border. The width is set in pixels, or by using one of the three pre-defined values: thin, medium, or thick. Note: The "border-width" property does not work if it is used alone. Use the "border-style" property to set the borders first. *Example* p.one {

border-style:solid; border-

width:5px;

}

p.two { border-
style:solid; border-
width:medium; }

Border Color

The border-color property is used to set the color of the border. Note: The "border-color" property does not work if it is used alone. Use the "border-style" property to set the borders first. *Example* p.one { border-style:solid; border-color:red;
}
p.two { border-
style:solid; border-
color:#98bf21;
}

Border - Individual sides

In CSS it is possible to specify different borders for different sides: *Example* p { border-top-style:dotted; border-right-style:solid; border-bottom-style:dotted; border-left-style:solid;
}

The example above can also be set with a single property: *Example*
border-style:dotted solid;
The border-style property can have from one to four values.

- border-style:dotted solid double dashed;
  ○ top border is dotted ○ right border is solid ○ bottom border is double
    ○ left border is dashed
- border-style:dotted solid double; ○ top border is dotted ○ right and left borders are solid ○ bottom border is double
- border-style:dotted solid; ○ top and bottom borders are dotted ○ right and left borders are solid

- border-style:dotted; ○ all four borders are dotted

The border-style property is used in the example above. However, it also works with border-width and border-color.

Border - Shorthand property

To shorten the code, it is also possible to specify all the border properties in one property. This is called a shorthand property. The shorthand property for the border properties is "border": *Example* border:5px solid red;

When using the border property, the order of the values are:

- border-width
- border-style
- border-color

It does not matter if one of the values above are missing (although, border-style is required), as long as the rest are in the specified order.

CSS Outlines

An outline is a line that is drawn around elements, outside the border edge, to make the element "stand out". The outline properties specifies the style, color, and width of an outline. However, it is different from the border property. The outline is not a part of the element's dimensions, therefore the element's width and height properties do not contain the width of the outline.

All CSS Outline Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	**Description**	**Values**	**CSS**
outline	Sets all the outline properties in one declaration	*outline-color* *outline-style outline-width* inherit	2
outline-color	Sets the color of an outline	*color_name* *hex_number* *rgb_number* invert inherit	2

outline-style	Sets the style of an outline	none dotted dashed solid double groove ridge inset outset inherit	2
outline-width	Sets the width of an outline	thin medium thick *length* inherit	2

CSS Margin

The CSS margin properties define the space around elements. The margin clears an area around an element (outside the border). The margin does not have a background color, and is completely transparent. The top, right, bottom, and left margin can be changed independently using separate properties. A shorthand margin property can also be used, to change all margins at once. Possible Values

Value	**Description**
auto	The browser sets the margin. The result of this is dependant of the browser
length	Defines a fixed margin (in pixels, pt, em, etc.)
%	Defines a margin in % of the containing element

It is possible to use negative values, to overlap content.

Margin - Individual sides

In CSS, it is possible to specify different margins for different

sides: *Example* margin-top:100px; margin-bottom:100px; margin-

right:50px; margin-left:50px;

Margin - Shorthand property

To shorten the code, it is possible to specify all the margin properties in one property. This is called a shorthand property. The shorthand property for all the margin properties is "margin": *Example* margin:100px 50px;

The margin property can have from one to four values.

- margin:25px 50px 75px 100px;
  ○ top margin is 25px ○ right
  margin is 50px ○ bottom
  margin is 75px ○ left margin is
  100px
- margin:25px 50px 75px; ○ top
  margin is 25px ○ right and left
  margins are 50px ○ bottom
  margin is 75px
- margin:25px 50px; ○ top and
  bottom margins are 25px ○
  right and left margins are 50px
- margin:25px; ○ all four margins
  are 25px

CSS Padding

The CSS padding properties define the space between the element border and the element content. The padding clears an area around the content (inside the border) of an element. The padding is affected by the background color of the element. The top, right, bottom, and left padding can be changed independently using separate properties. A shorthand padding property can also be used, to change all paddings at once.

Possible Values

Value	**Description**
length	Defines a fixed padding (in pixels, pt, em, etc.)

%	Defines a padding in % of the containing element

Padding - Individual sides

In CSS, it is possible to specify different padding for different sides: *Example* padding-top:25px; padding-bottom:25px; padding-right:50px; padding-left:50px;

Padding - Shorthand property

To shorten the code, it is possible to specify all the padding properties in one property. This is called a shorthand property. The shorthand property for all the padding properties is "padding": *Example* padding:25px 50px;

The padding property can have from one to four values.

- padding:25px 50px 75px 100px;
  ○ top padding is 25px ○ right padding is 50px ○ bottom padding is 75px ○ left padding is 100px

- padding:25px 50px 75px; ○ top padding is 25px ○ right and left paddings are 50px ○ bottom padding is 75px

- padding:25px 50px; ○ top and bottom paddings are 25px ○ right and left paddings are 50px

- padding:25px; ○ all four paddings are 25px

Chapter Review Questions

1. Define what is CSS
2. Outline the advantages of CSS
3. Giving examples can you explain three ways to insert CSS into a HTML document
4. Write CSS code to add a background image to a HTML document
5. Write CSS code to display the following box model

Suggested Further Reading

1. Niederst J., Learning Web design, Shroff Publishers
2. Peter, K., Web design, Tools and Techniques, Peach Pit press
3. http://www.w3schools.com